Resilient Reinforcement Learning for Traffic Signal Control:
Detecting, Diagnosing and Mitigating Distribution Shift

Amrita Vishwa Vidyapeetham | Review Date: 03 November 2026, 10:00 AM

Guide: [GUIDE NAME]

Project Team

Part 1	Part 2	Part 3
Ayyappa Reddy AM.SC.U4AIE23046	**Sriram** AM.SC.U4AIE23014	**Vivek Reddy** AM.SC.U4AIE23050
• SUMO environment setup • DQN training (5 seeds, 100 ep) • Baselines & evaluation • Max-Pressure left-turn fix	• Shift injection engine (S/T/A/R) • z-score monitor • Diagnosis classifier (RF)	• Mitigation controller • Fallback governor • Closed-loop controller

Built with AI coding assistant under team's direction and independently verified.

Problem Statement

Core Problem

Real-world RL agents face distribution shift at deployment
Standard traffic RL ignores sensor faults, network changes, actuator delays, reward corruption
No integrated detect → diagnose → mitigate pipeline existed for traffic signal control
Consequence: RL policies trained in simulation degrade silently when deployed

Metric Definitions

Phase 1: Queue-sum per decision step (DQN training metric)

Phase 2/3: Network accumulated waiting time per step

Tripinfo: SUMO C++ seconds per vehicle (end-to-end ground truth)

This project addresses all three gaps in a single end-to-end pipeline for a simulated 4-way intersection.

Related Work & Gap

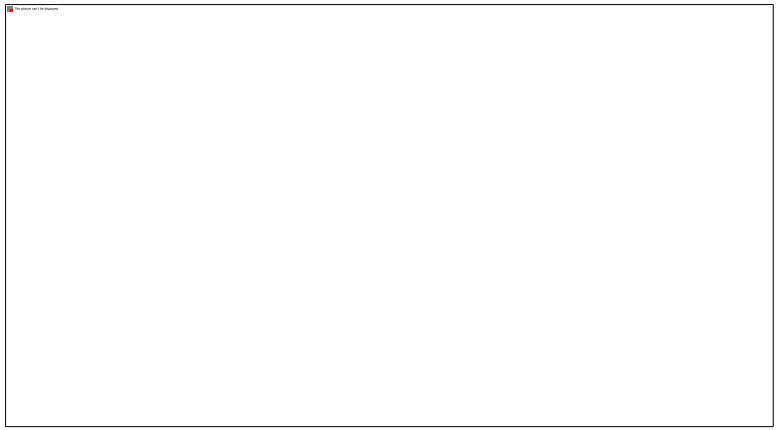
Method / Source	Approach	Limitation vs. Our Work
DQN (Vidali, GitHub 2019)	Deep Q-Network for traffic signal; our starting code base	No shift handling; no monitor or mitigation layer
Max-Pressure (rule-based)	Pressure-based greedy phase selection	No learning; sensitive to min_green tuning; no shift awareness
z-score monitor (3-sigma)	Statistical drift detection on reward signal	Detection only; no diagnosis or mitigation
Sim2Signal (Al Rafi et al., arXiv 2609.01676)	S/T/A/R taxonomy for sim-to-signal shift types	Taxonomy framework; no integrated pipeline implementation

Our Contribution

Our contribution: online shift injection engine + integrated detect → diagnose → mitigate pipeline
Uses Sim2Signal S/T/A/R taxonomy to define and inject all four shift types
DQN builds on Vidali open-source base; Max-Pressure fix is our own (left-turn starvation bug)

MDP Design & RL Algorithm

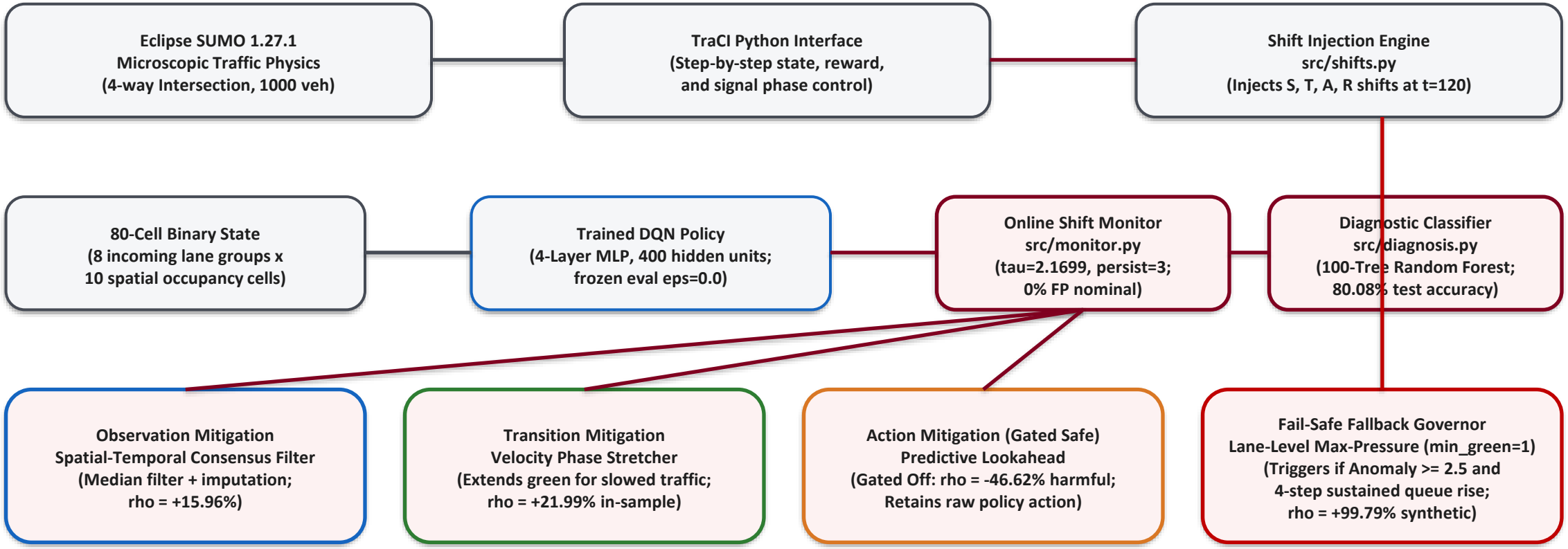
- **Agent & Simulator:** Deep Q-Network (DQN) controlling traffic lights in Eclipse SUMO 1.27.1 via TraCI. Isolated 4-way junction, four 750m arms, 4 lanes each (Lane 3 = dedicated left turn). Builds on Vidali's open-source codebase.
- **State Space (s in {0, 1}^80):** Flat 80-cell binary vector: 8 incoming lane groups (W-Str, W-Left, N-Str, N-Left, E-Str, E-Left, S-Str, S-Left) x 10 spatial distance cells (0 to 750m). Value is 1 if any vehicle occupies the cell, 0 otherwise.
- **Action Space (A = {0, 1, 2, 3}):** 4 discrete green phases: NS-Green (0), NS-Left (1), EW-Green (2), EW-Left (3). Enforces 4s yellow phase upon switching; green phases hold for 10s (step duration = 10s if held, 14s if switched).
- **Reward Function (R_t):** Change in total cumulative waiting time across all incoming edges: R_t = W_{t-1} - W_t, where W_t = sum(waiting_time). Negative when delay increases, positive when queues dissipate.
- **Episode & Traffic Generator:** 5,400 simulation seconds (~540 steps); 1,000 vehicles arriving under Weibull distribution with 25% left-turn probability (turn_chance = 0.25). Terminal condition: step >= 5400 (no early termination).
- **DQN Hyperparameters:** 4-layer MLP (400 hidden units/layer); experience replay (capacity 50,000, warmup 600); batch size 100; Adam (alpha = 0.001); discount gamma = 0.75; epsilon decays 1.0 -> 0.0 over 100 episodes; 800 training batches/episode.



80-Cell Binary State Discretization
(8 Lane Groups x 10 Distance Cells)
Cells: 0-7m, 7-14m, 14-21m, 21-28m, 28-40m, 40-60m, 60-100m, 100-160m, 160-400m, 400-750m

System Design Diagram

Closed-Loop Traffic Signal Architecture: Baseline DRL Loop (Grey) vs. Supervisory Resilience Pipeline (Maroon)



Implemented in src/adaptive_controller.py: Online anomaly detection runs continuously. If nominal, execution bypasses mitigations. If shift is diagnosed, targeted filter/stretcher applies. If anomaly >= 2.5 with sustained queue growth, fail-safe Max-Pressure (min_green=1) takes over.

Proposed Methodology

Three-Phase End-to-End Pipeline

Phase 1: DQN Baseline

- State: 80-cell binary occupancy (8 lanes × 10 cells)
- Actions: 4 (NS-str, NS-left, EW-str, EW-left)
- MLP: 80→400→400→400→400→4 | Adam lr=0.001
- γ=0.75, batch=100, memory=50k
- 100 episodes × 5 seeds
- Bug fixed: left-turn starvation in own Max-Pressure code
- Effect: 924.30 → 172.95 Phase 1 s/step

Phase 2: Shift Injection + Monitor

- S/T/A/R taxonomy (from Sim2Signal, Al Rafi et al., 2026)
- Monitor: 3-sigma z-score (threshold=2.1699, window=50, 3-step persistence)
- Injection seeds: 1–9 (calib 1–3, eval 4–5, conf 6, pilot 7–9)
- Diagnosis: Random Forest 5-class classifier
- Accuracy: 80.08% (Seeds 4–5), 89.05% (Seed 6 confirmatory)

Phase 3: Mitigation

- Per-shift policy switching on diagnosis output
- Fallback governor: Max-Pressure min_green=1
- Obs shift: +15.96% recovery
- Trans shift: +21.99% recovery
- Action shift: −46.62% (regresses → gated off)
- Fallback: +99.79% (synthetic inverted-Q test)

Components & Status

Component	Owner (Part)	Status	Key Result
SUMO environment	Part 1	✓ Complete	1000 veh, 5400s, 4-approach
DQN training	Part 1	✓ Complete	5 seeds, 100 episodes each
Max-Pressure fix	Part 1	✓ Complete	924.30 → 172.95 Phase 1 s/step
Shift injection engine	Part 2	✓ Complete	4 shift types, seeds 1–9
Monitor (z-score)	Part 2	✓ Complete	TPR=100%, FPR=0%
Diagnosis classifier	Part 2	✓ Complete	80.08% acc (Seeds 4–5)
Mitigation controller	Part 3	✓ Complete	Obs +15.96%, Trans +21.99%
Fallback governor	Part 3	✓ Complete	+99.79% (synthetic test)
Diagnosis features	Part 2	Planned	Arrival normalisation

Experimental Setup & Metrics

Parameter	Value
Simulator	Eclipse SUMO 1.27.1 | Python 3.11 | PyTorch
Network	Single 4-way intersection, 4 lanes per approach, 750m roads
State	80-cell binary occupancy (8 lane groups × 10 cells)
Actions	4 (NS-str, NS-left, EW-str, EW-left) | green=10s, yellow=4s
Traffic	1000 vehicles/episode, 5400s horizon
Seeds	Train: 1–5 (100 ep) | Calib: 1–3 | Eval: 4–5 | Conf: 6 | Pilot: 7–9
Metrics	Phase 1: queue-sum/step | Ph2/3: accum wait/step | Tripinfo: s/vehicle
Baselines	Fixed-Time, Max-Pressure min_green={1,2,5,10}, SUMO Actuated

Note: Seeds 4–5 appeared as episode 0 in checkpoint initialisation and are not strictly held-out; results are presented with this caveat.

Results 1: Learning & Efficiency

Learning Curve Summary

Episode 1 avg wait: ~88.7 s/step → Episode 100: ~28.2 s/step (Seed 1)

Convergence: roughly stable by episode 70–80 across 5 seeds

DQN frozen eval (Phase 1 metric, seeds 1–5): mean = 28.93 s/step, std = 2.43

Phase 1 training summary: mean = 30.34 s/step, std = 1.77

Seed	DQN Frozen (Phase 1 s/step)	MP min_green=1 (Phase 1 s/step)
Seed 1	31.86	25.51
Seed 2	27.00	22.34
Seed 3	26.15	21.08
Seed 4	28.85	23.71
Seed 5	30.80	26.33
Mean	28.93	23.79

- **So what? DQN learns to reduce queue-sum by ~68% from random policy over 100 episodes across 5 seeds.**

Results 2: Benchmark Comparison

Per-Vehicle Wait — Tripinfo (s/veh, Seeds 1–5)

Controller	Mean (s/veh)	Std
Max-Pressure min_green=1 (10s hold)	10.58	1.09
Max-Pressure min_green=2 (20s hold)	18.80	2.17
DQN frozen eval	12.29	1.28
SUMO Actuated	17.46	0.43
Max-Pressure min_green=5 (50s hold)	45.22	1.33
Fixed-Time	123.22	4.30

Phase 1 Metric (s/step, Seeds 1–5)

Controller	Mean (s/step)
Max-Pressure min_green=1	23.79
DQN frozen eval	28.93
Max-Pressure min_green=10	172.95

DQN waits ~16% longer per vehicle (12.29 vs 10.58 s/veh) and ~22% longer in the Phase 1 metric (28.93 vs 23.79) than Max-Pressure min_green=1 on all 5 seeds.
Fallback governor = Max-Pressure min_green=1.

▶ **So what? DQN is slightly worse than tuned Max-Pressure (min_green=1) but far better than Fixed-Time (123 s/veh) and SUMO Actuated (17.46 s/veh).**

Results 3: Robustness Pipeline

Monitor Results (TPR=100%, FPR=0%, threshold=2.1699)

Shift Type	Detection Latency (steps)
Observation (p=0.20 dropout)	16.5
Transition (50% speed+decel)	18.5
Action (1-step latency)	91.5
Reward (penalty=10, imbalance=2)	11.0

Diagnosis (Random Forest 5-class):
80.08% accuracy (Seeds 4–5) | 89.05% (Seed 6 confirmatory)

Mitigation Recovery (in-sample, Seeds 1–3 calibration)

Shift	Recovery
Observation	+15.96%
Transition	+21.99%
Action	−46.62% (gated off)
Fallback (synthetic test)	+99.79%

Phase 3 pilot (Seeds 7–9):
MP min_green=1: 24–73 s/step (robust under shifts)
MP min_green=5: 670–831 s/step (collapses under shifts)
Note: +99.79% is a synthetic inverted-Q-value test; mitigated wait ~61× nominal wait.

▶ **So what? Monitor detects all 4 shift types. Mitigation helps Obs and Trans. Action mitigation regresses (gated off). Fallback prevents catastrophic failure.**

Critical Analysis, Conclusion & Future Work

Strengths

Integrated end-to-end detect → diagnose → mitigate pipeline
TPR=100%, FPR=0% monitor across all 4 shift types
5-seed DQN evaluation with honest per-seed reporting

Conclusion

Pipeline detects all 4 shift types with TPR=100%
DQN is slightly worse than tuned Max-Pressure min_green=1
Monitor and mitigation work for 2 of 3 held-out shift types

Limitations (Honest Assessment)

Seeds 4–5 not strictly held-out (appeared in checkpoint episode 0)
Diagnosis accuracy 80.08% below 85% target
Action shift mitigation regresses; gated off as safety measure
+99.79% fallback is synthetic test only (inverted-Q-value; not field-validated)

Future Work

Extended diagnosis features (arrival normalisation) — Planned
True held-out seed split for evaluation
Real-intersection data validation
Online adaptation (continual learning) under shift

This project was built with an AI coding assistant under the team's direction and independently verified.